Artificial Neural Networks of The Perceptron, Madaline, and Backpropagation Family

Bernard Widrow Micheal A. Lehr

Presented by Xuanchong Li

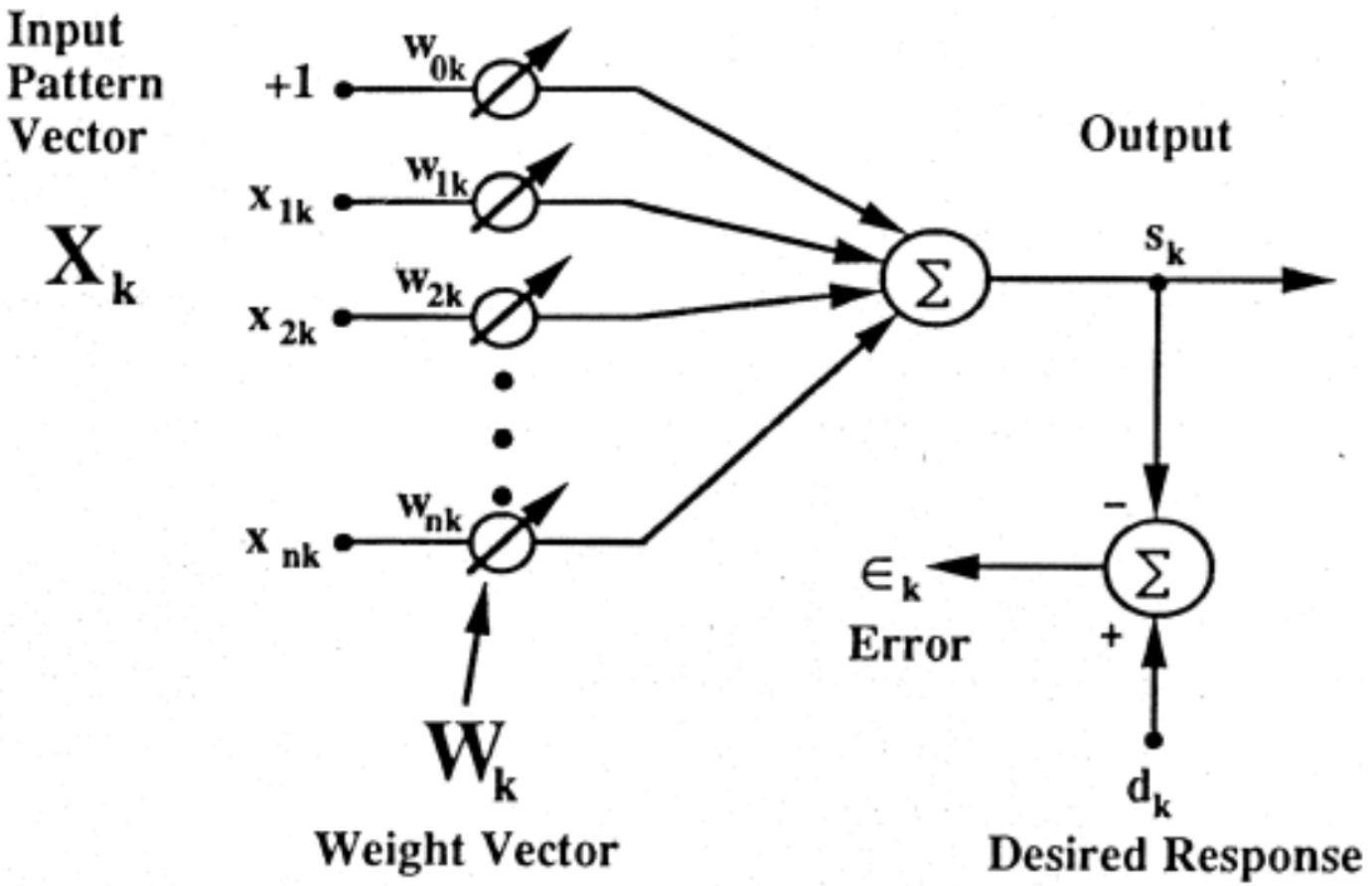
Outline

Algorithms History 1960s - 1990s

- 1960: Least Mean Square (LMS) algorithm (Widrow and his student), Perceptron rule (Rosenblatt)
- Mid 1960s: Madaline (Multiple adaptive linear elements) rule I (MRI) and application in speech, weather forecasting, pattern recognition (Widrow and his student)
- 1971: Backpropagation (Werbos). It was first ignored by community, then re-discovered in 1982 by Parker, finally became famous with work of Rumehart, Hinton, and Williams.
- 1987: Madaline Rule II (MRII) by Widrow and his student. The goal is for adapting multiple players network
- 1988: Madaline Rule III (MRIII) by David Andes. Widrow and his student found it is mathmatically equivalent to backpropagation

Outline

Adaptive linear combiner



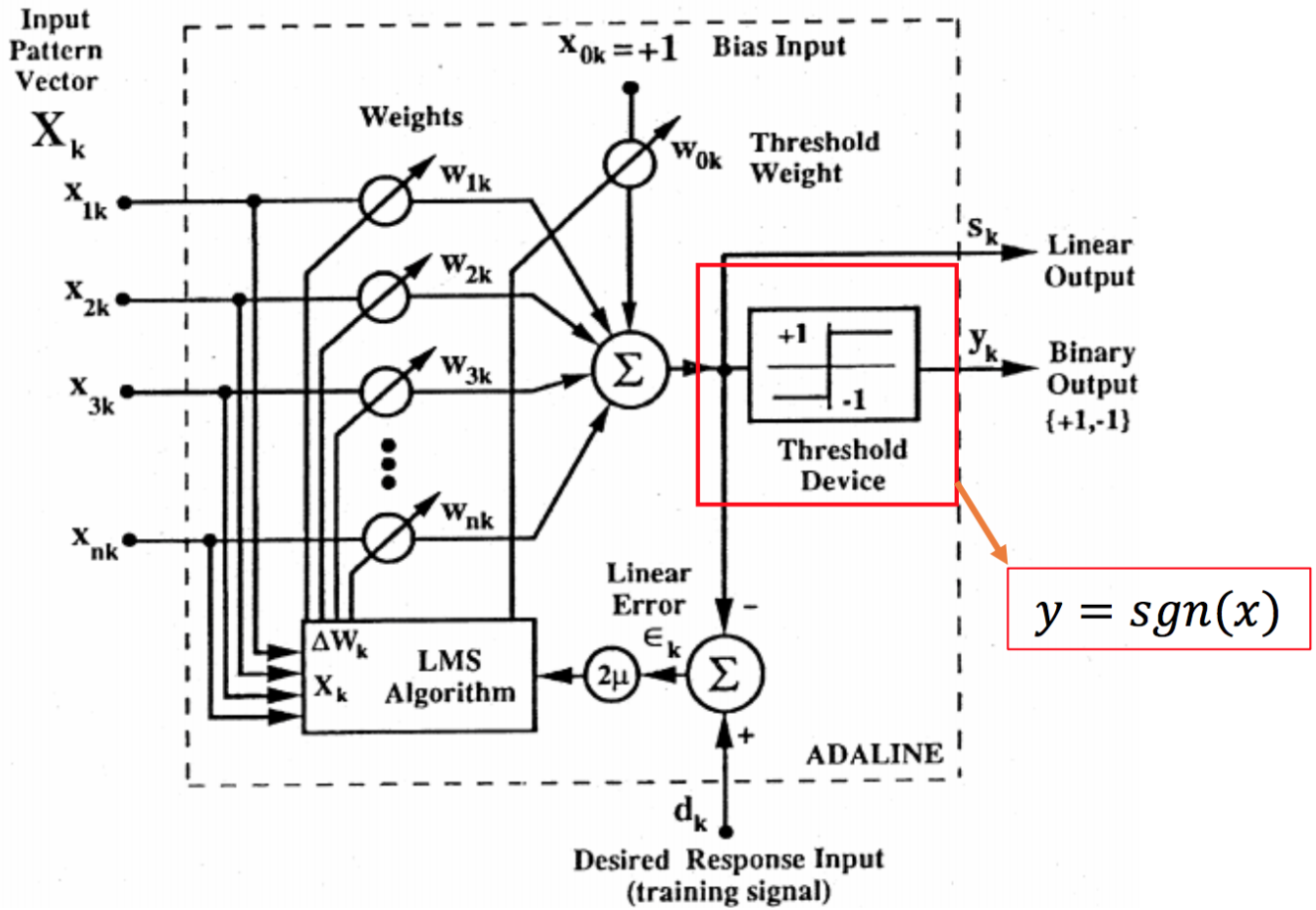
$$X_k=[x_0, x_{1k}, x_{2k},\ldots, x_{nk}]^T$$

$$W_k=[w_{0k}, w_{1k}, w_{2k},\ldots, w_{nk}]^T \qquad s_k=X_k^T W_k$$

Outline

Adaptive linear element (Adaline)

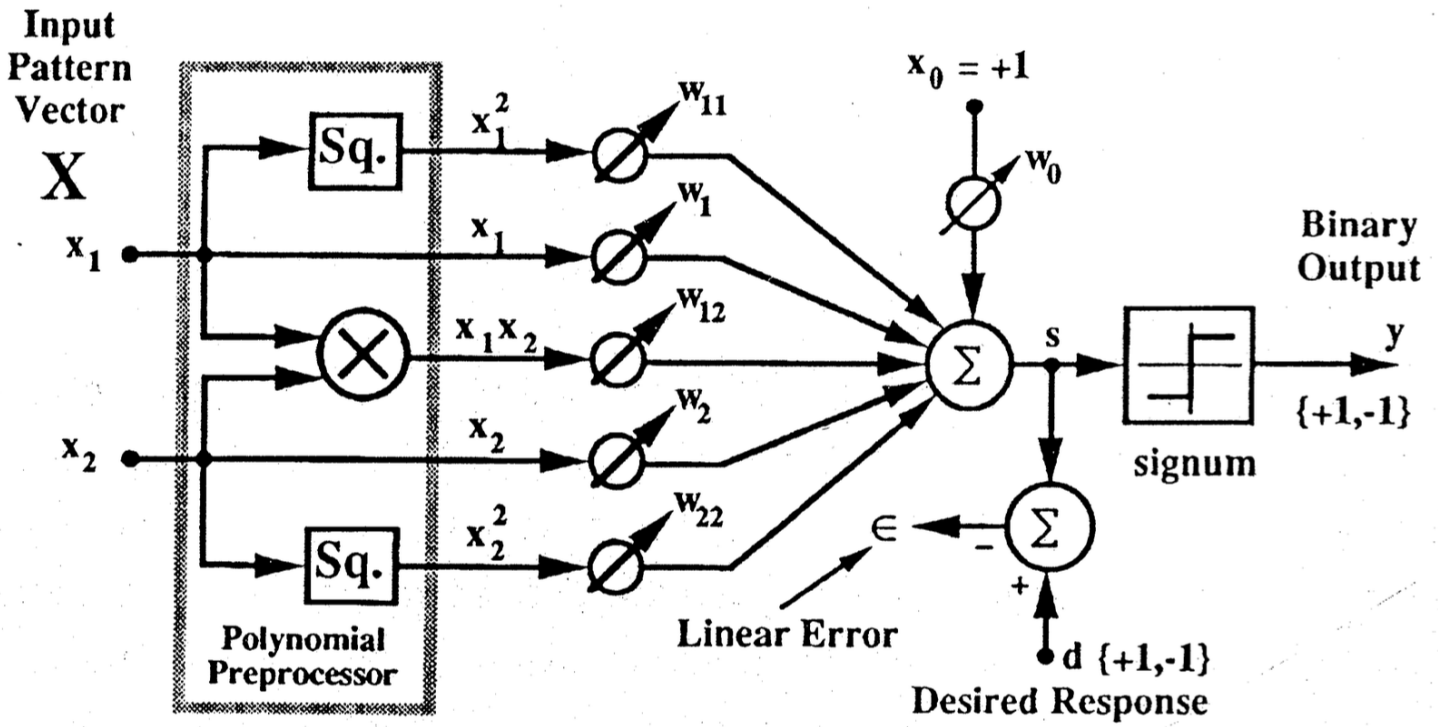
- Basic building block used in neural networks
- Adaptive threshold logic element: an adaptive linear combiner + hard-limiting quantizer



Outline

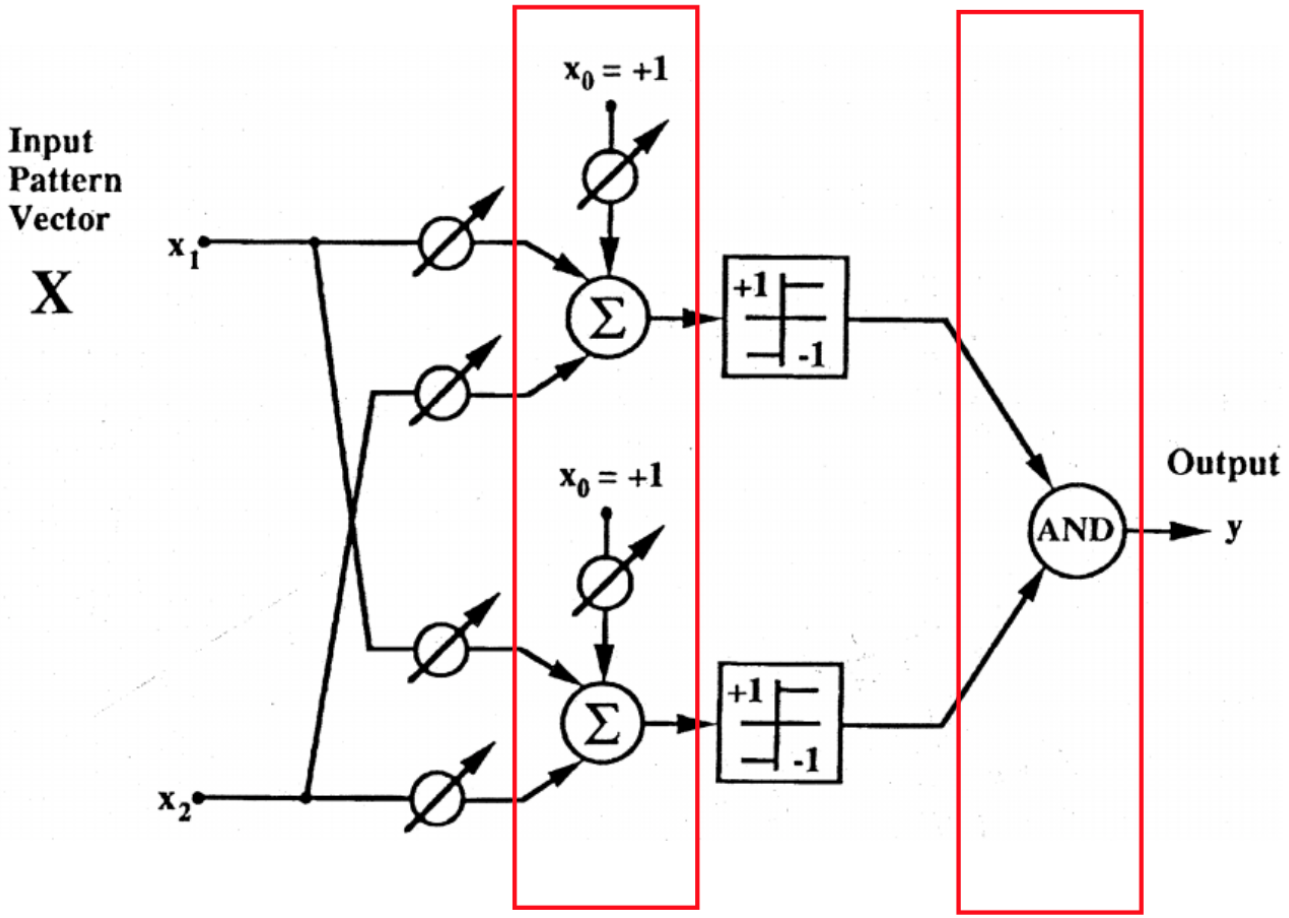
Polynomial Preprocessor

- Fixed preprocessing network + a single adaptive element
- The choice of preprocessing function matters a lot



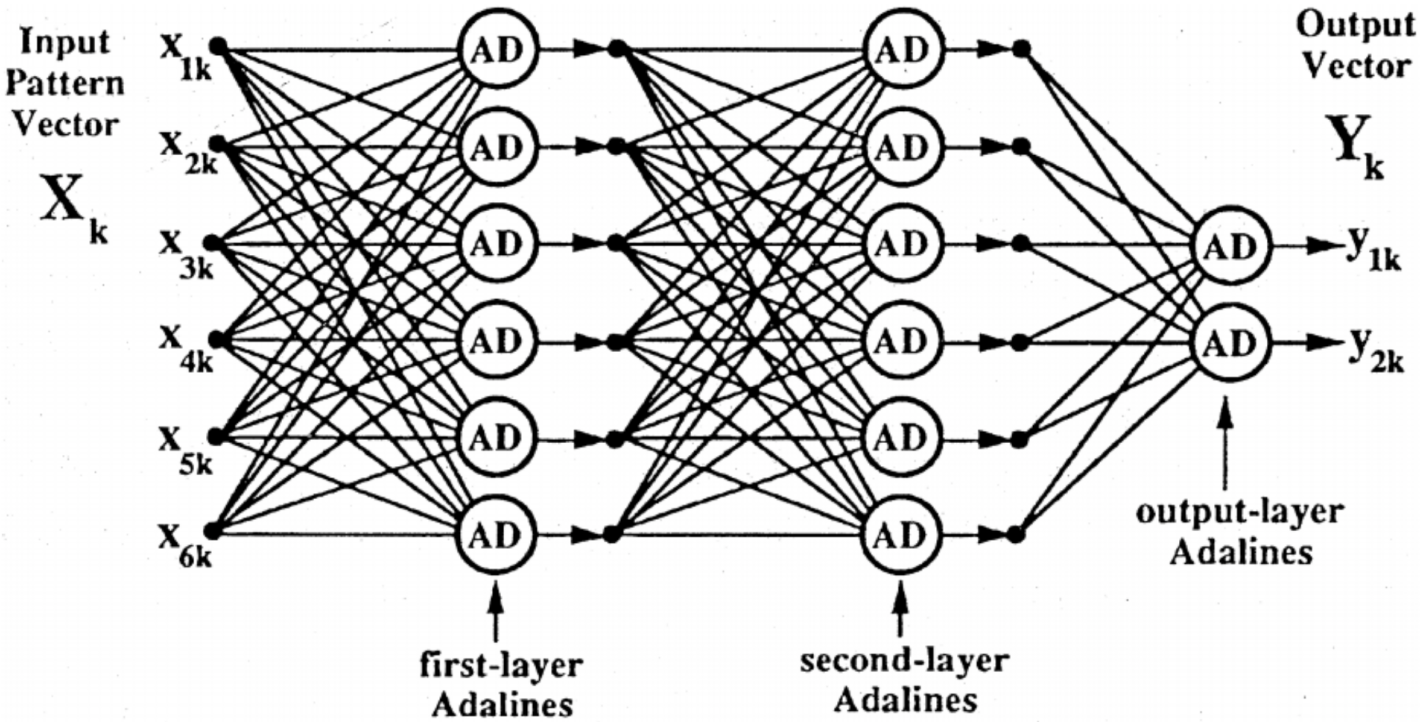
Madaline I

- One of the earliest trainable layered neural networks.
- A layer of ADALINE + fix logic device (AND, OR, MAJ)



Feedforward Network

- All layers are adaptive
- Exp. a fully-connected three-layer feedforward adaptive network



Outline

The Minimal Disturbance Principle

- Minimal Disturbance Principle: Adapt to reduce the output error for the current training pattern, with minimal disturbance to responses already learned.
- It is behind every learning algorithm in the paper.

Two Classes of Rules

- Error correction rules: alter the weights of a network to correct a certain proportion of the error in the output response to the present input pattern
- Steepest descent rules: alter the weights during each pattern presentation by gradient descent with the objective of reducing mean-square-error, average over all training patterns

Outline

Linear Rules

α-LMS Algorithm

- Linear Rule: alter the weights of the adaptive threshold element with each pattern presentation to make an error correction which is proportional to the error itself.
- Follows Error Correction Rule
- Weight update rule:

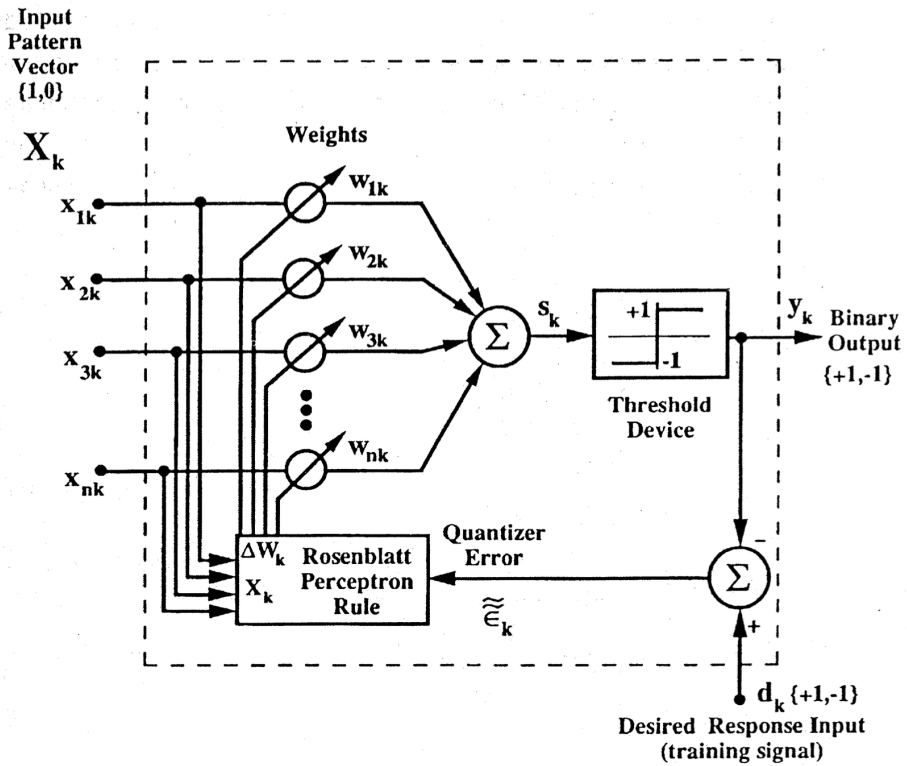
$$W_{k+1} = W_k + \alpha \frac{\epsilon_k X_k}{|X_k|^2}$$

- Error at k: $\epsilon_k = d_k - W_k^T X_k$
- Error Change: $\Delta\epsilon_k = -\alpha\epsilon_k$
- Weights are usually initialized as 0.
- Learning Rate: $0.1 < \alpha < 1$, controls stability and convergence rate.
- Self-normalizing: choice of α does not depend on the magnitude of the input signals.

Non-linear Rules

Perceptron

- A non-linear algorithm: weights change is collinear with the input pattern vector and the linear error.
- Follows Error Correction Rule
- Weight update rule: $W_{k+1} = W_k + \alpha \frac{\tilde{\tilde{\epsilon}}_k}{2} X_k$



α-LMS V.S. Perceptron

- α Value
  - α-LMS: controls stability and speed of convergence
  - Perceptron rule: does not affect the stability of the perceptron algorithm, and it affects convergence time only if the initial weight vector is nonzero
- Binary or continuous response
  - α-LMS: both binary and continuous response
  - Perceptron: only binary
- Linearly separable training patterns
  - α-LMS: may fail to separate linearly separable set
  - Perceptron: separate any linearly separable set
- Nonlinearly separable training patterns
  - α-LMS: does not lead to unreasonable weight solution
  - Perceptron: goes on forever is not linearly separable, and often does not yield a low-error solution. Usually end up with a small norm weight vector.

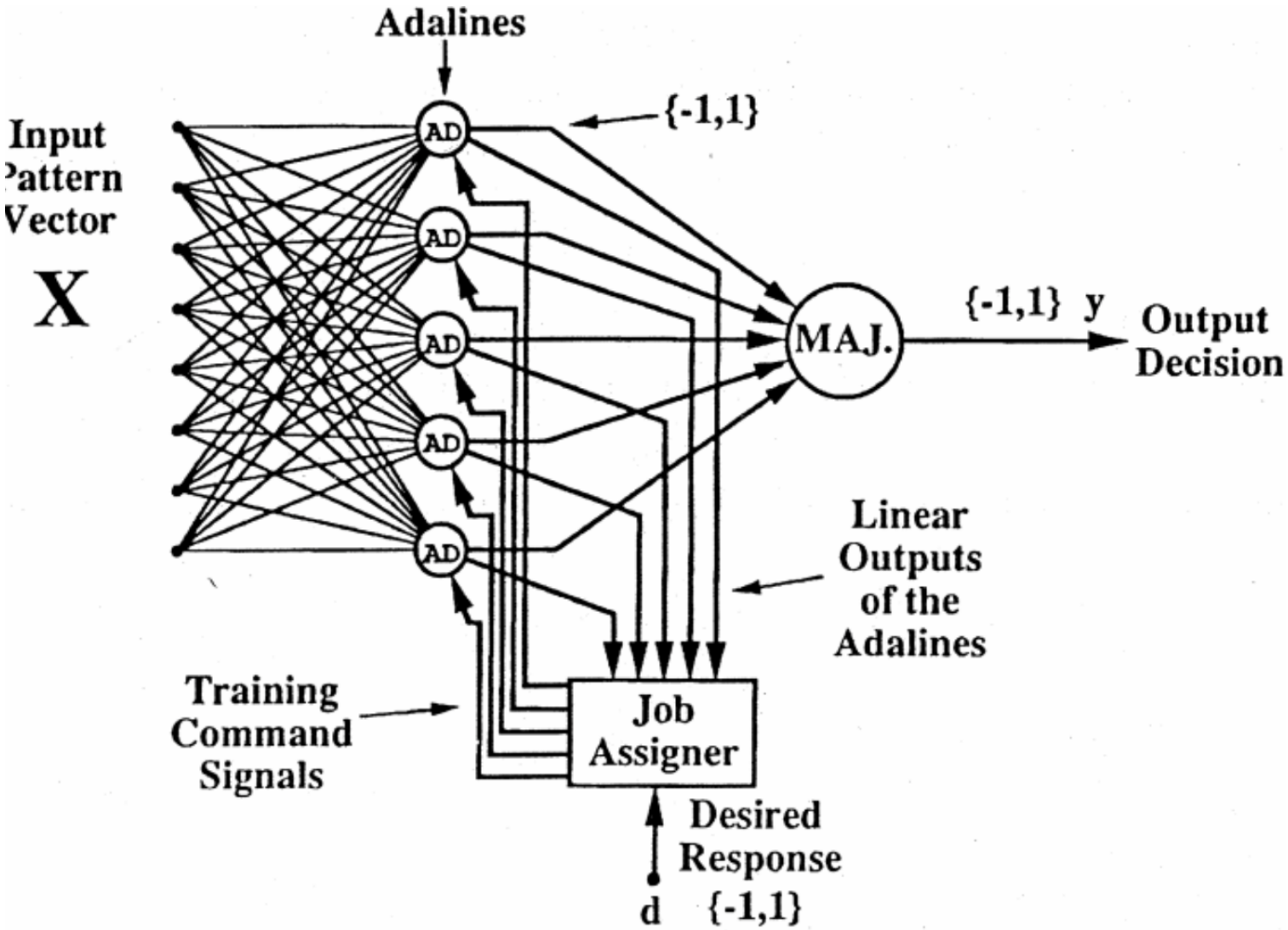
May's Algorithm

$$\mathbf{W}_{k+1} = \begin{cases} \mathbf{W}_k + \alpha \, \tilde{\tilde{\epsilon}}_k \, \dfrac{\mathbf{X}_k}{2|\mathbf{X}_k|^2} & \text{if } |s_k| \geq \gamma \\ \mathbf{W}_k + \alpha d_k \dfrac{\mathbf{X}_k}{|\mathbf{X}_k|^2} & \text{if } |s_k| < \gamma \end{cases}$$

- Non-linear error correction rule
- Introduce the "deadzone" γ
- Separate any linearly separable set; For nonlinearly separable set, Mays rule performs much better than Perceptron rule because a sufficiently large dead zone tends to cause the weight vector to adapt away from zero when any reasonably good solution exists

Multi-element Networks

Madaline Rule I (MRI)

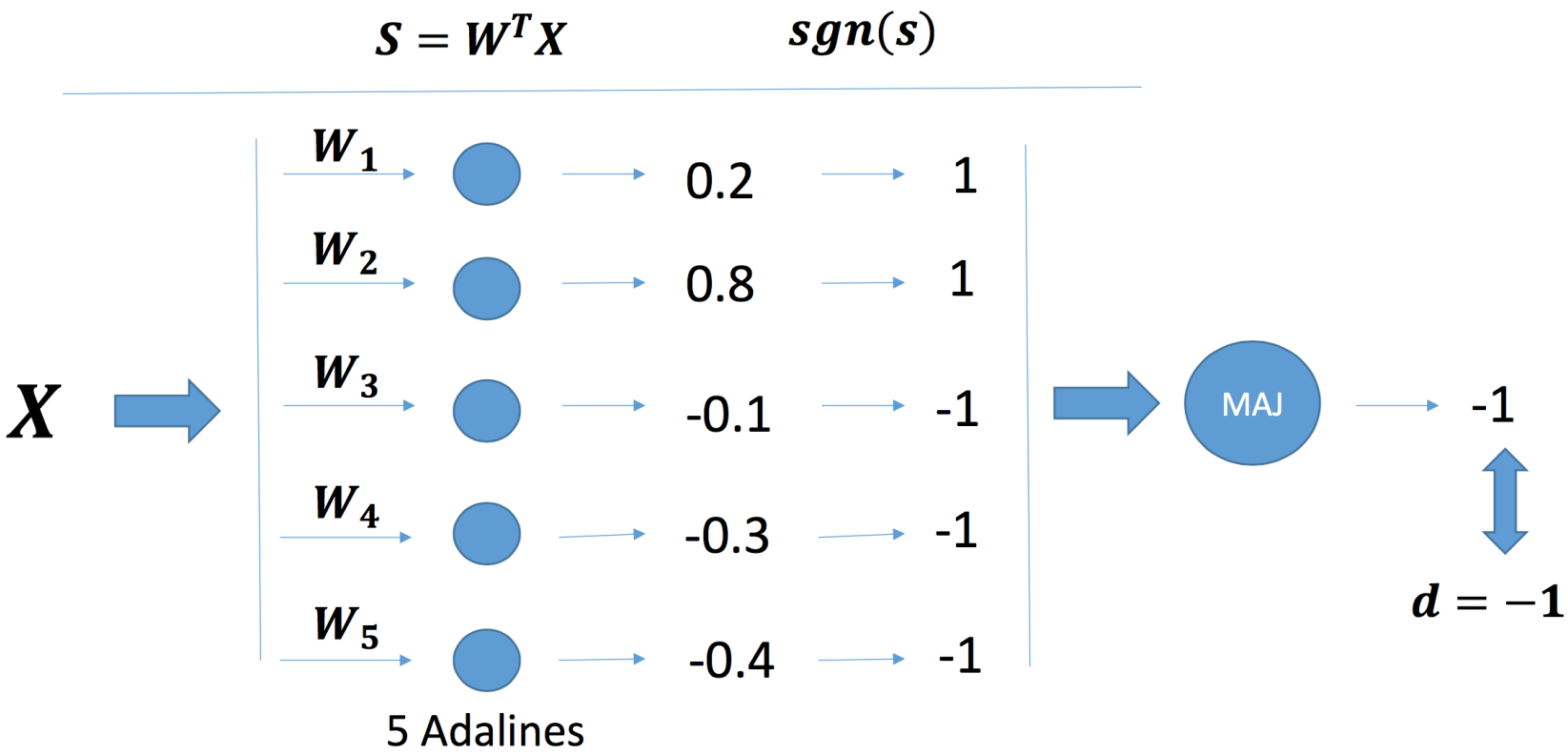


Multi-element Networks

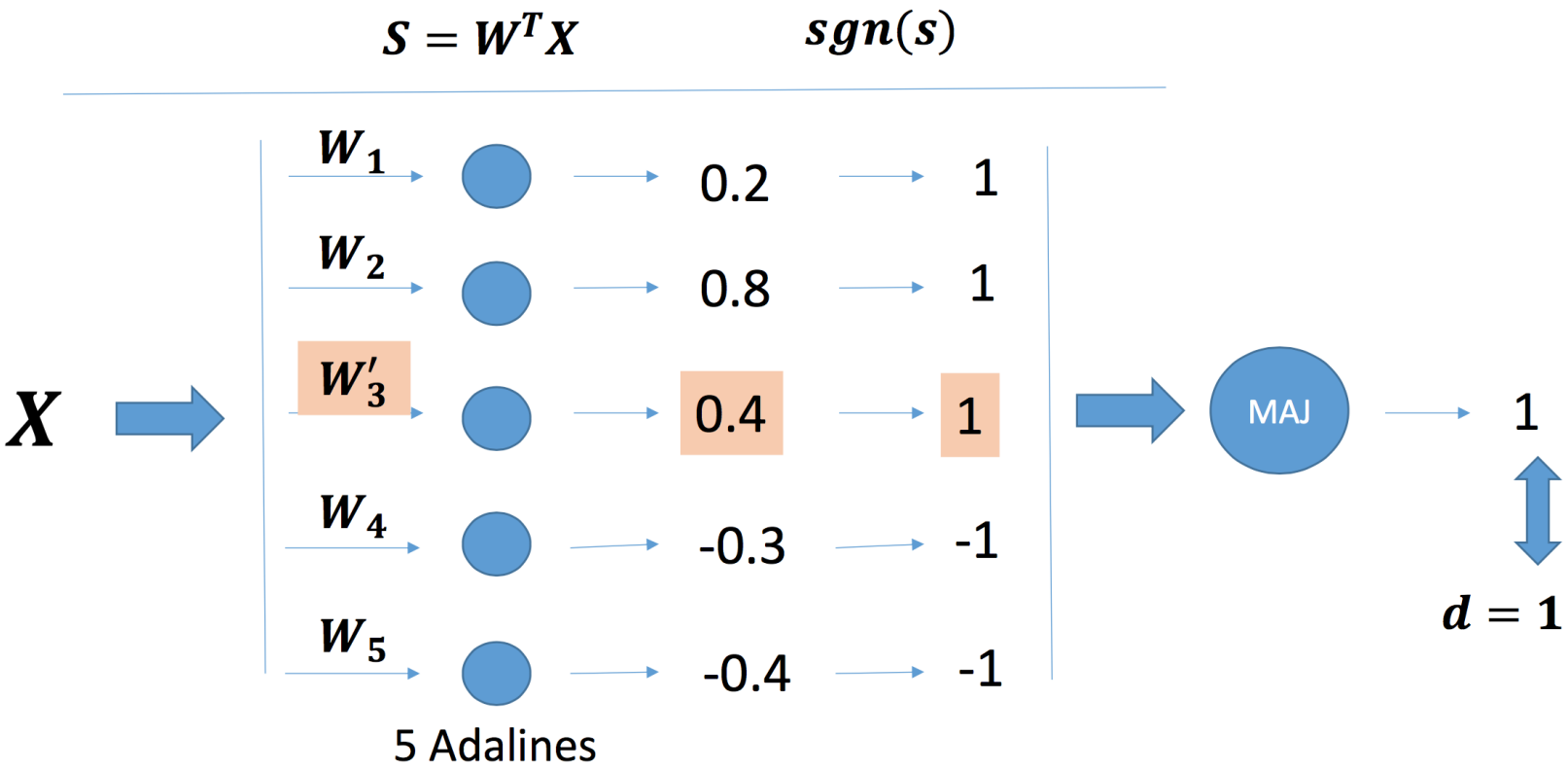
Madaline Rule I (MRI)

- If an error happens, pick the ADALINE with smallest $|s_k|$ to adapt.j
- Weights are initially set to small random values
- Weight vector update: can be changed aggressively using absolute correction or can be adapted by the small increment determined by the α-LMS algorithm
- Principle: assign responsibility to the Adaline or Adalines that can most easily assume it
- Pattern presentation sequence should be random

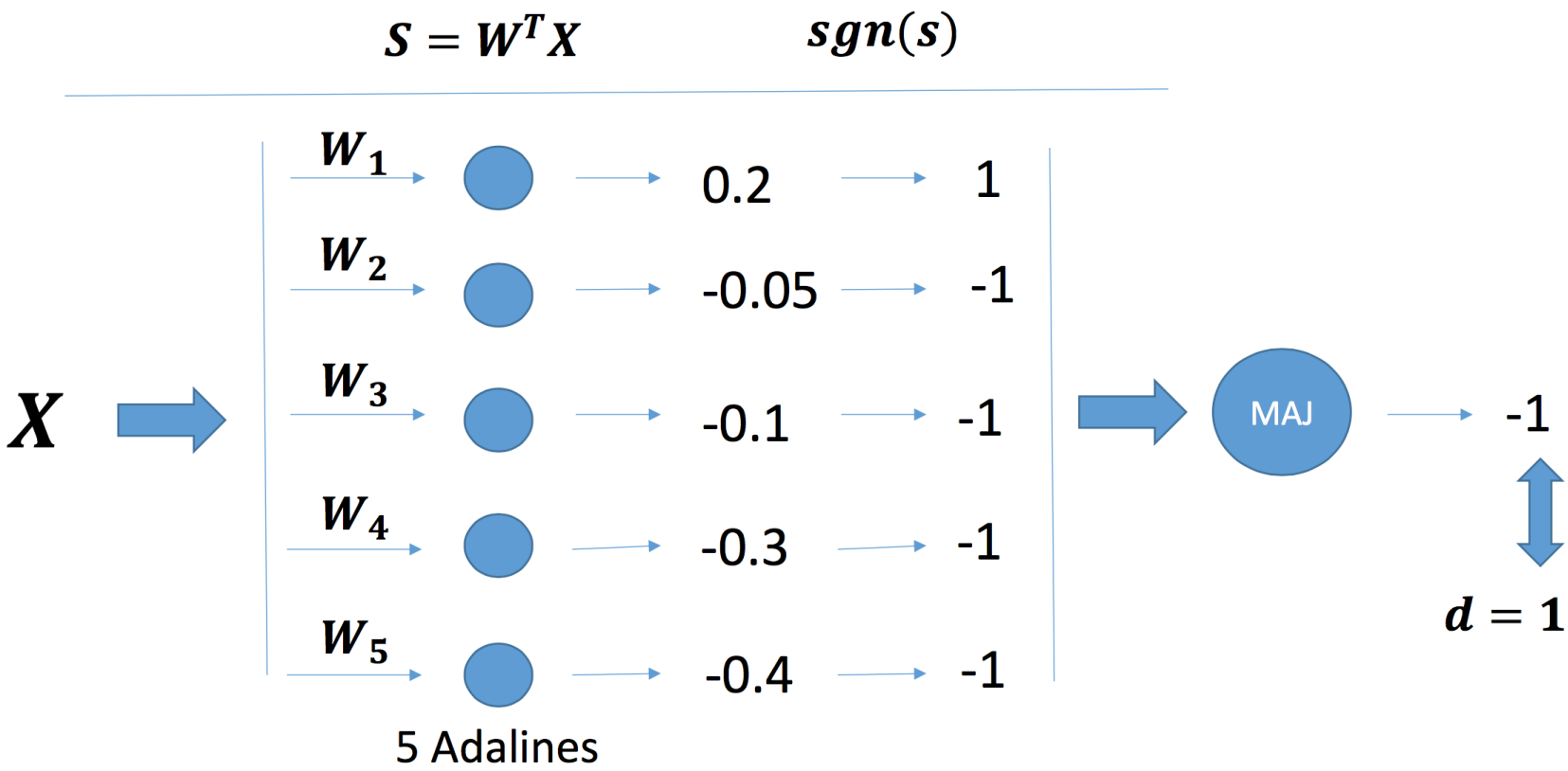
Madaline Rule I (MRI) example



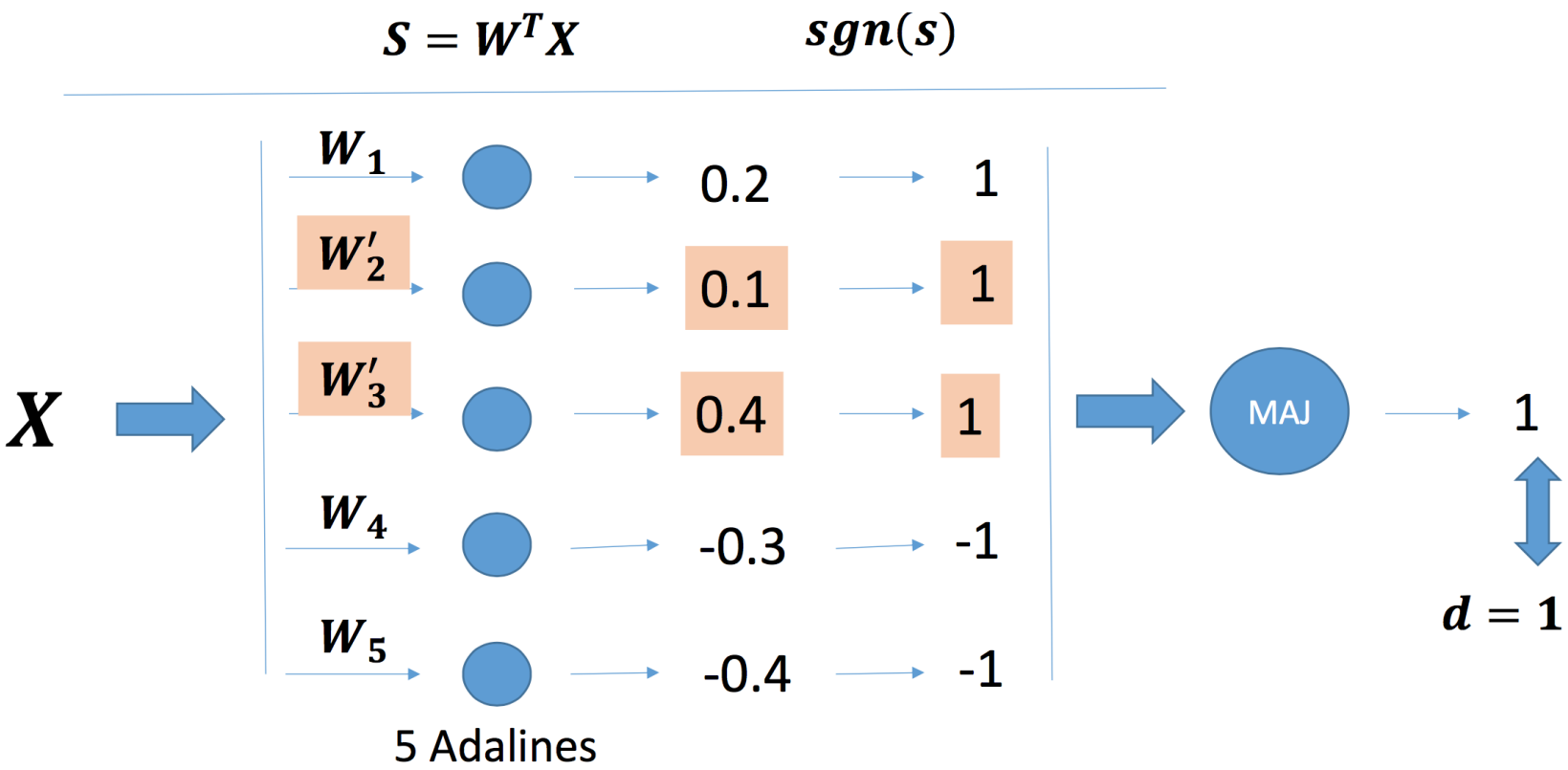
Madaline Rule I (MRI) example



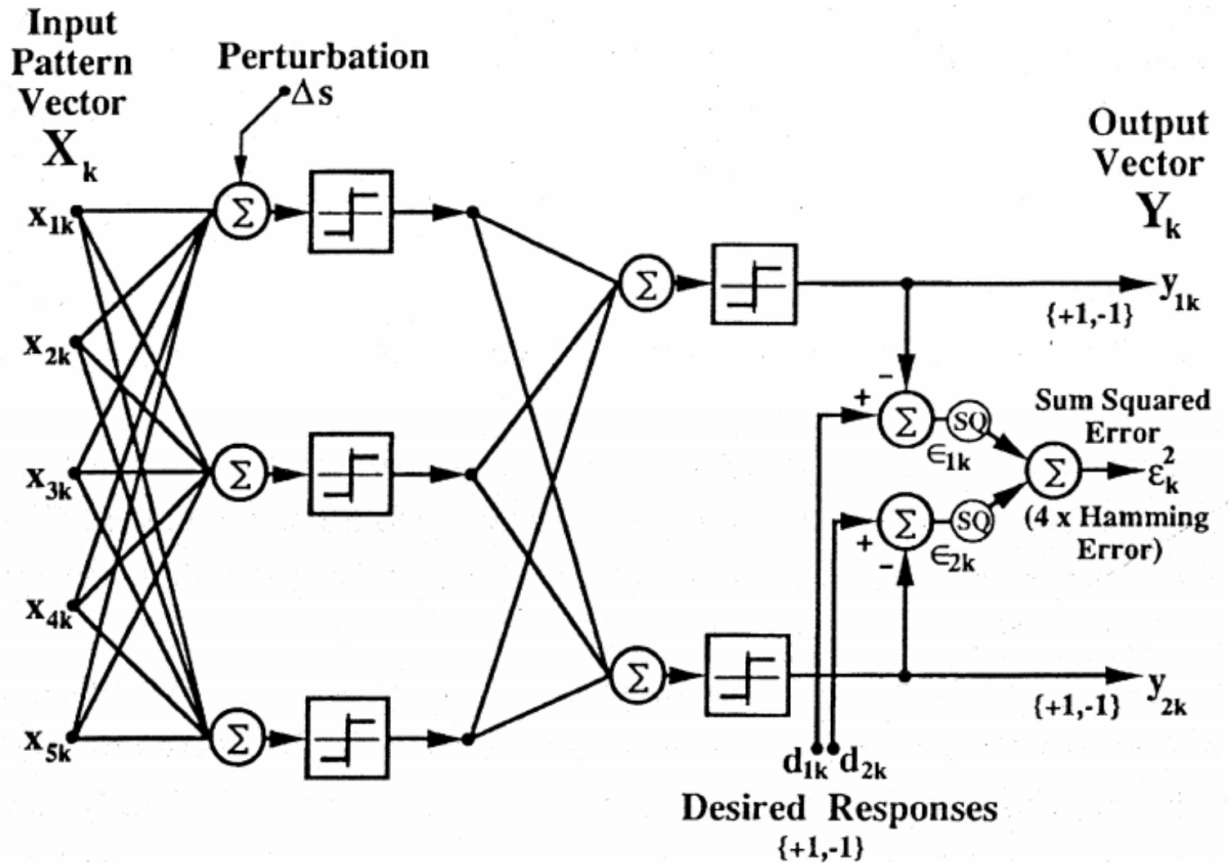
Madaline Rule I (MRI) example



Madaline Rule I (MRI) example



Madaline Rule II (MRII)



Typical two-layer Madaline II architecture

Madaline Rule II (MRII)

- Weights in both layers are adaptive
- Weights are initially set to small random values
- Random pattern presentation sequence
- Adapting the first-layer Adalines
  - Select the smallest linear output magnitude
  - Perform a "trial adaption" by adding a Δs perturbation of suitable amplitude to invert its binary output
  - If the output error is reduced, remove Δs, and change the weight of the select Adaline using α-LMS algorithm
  - Perturb and update other Adalines in the first layer with sufficiently small s_k output
  - After exhausting possibilities with the first layer, move on to next layer and proceed in a like manner
  - Random select a new training pattern and repeat the procedure.

Steepest Descent Rules

Single Threshold Element

- Objective of adaptation: reduce error averaged over the training set rather than reducing a given proportion of the error in each presentation of training data.
- The most common error: mean-square-error (MSE)
- $W_{k+1} = W_k + \mu(-\bigtriangledown_k)$. μ: controls stability and convergence speed.
- In practice, it works with on data at a time. It minimizes MSE approximately.

Steepest Descent Rules

Linear Rules

- The steepest descent rule is linear if weight changes are proportional to the linear error
- Linear Combiner:
  $\epsilon_k^2 = (d_k - X_k^T W_k)^2 = d_k^2 - 2 d_k X_k^T W_k + W_k X_k X_k^T W_k$
- MSE: $E[\epsilon_k^2] = E[d_k^2] - 2E[d_k X_k^T] W_k + W_k^T E[X_k X_k^T] W_k$
- $P^T \triangleq E[d_k X_k^T]$, $R \triangleq E[X_k X_k^T]$
- $\bigtriangledown_k = \frac{\partial E[\epsilon_k^2]}{\partial W_k} = -2P + 2RW_k$
- Set the gradient to zero: $W^* = R^{-1}P$ (Wiener weight vector)
- Need to compute R^{-1} and P?

Steepest Descent Rules

μ-LMS Algorithm

- Obtain accurate estimate of W^* without computing R^{-1} and P
- Use instantaneous gradient $\hat{\nabla}_k = \frac{\partial \epsilon_k^2}{\partial W_k}$. It is a unbiased estimate of the true gradient.
- Weight update: $W_{k+1} = W_k + \mu(-\hat{\nabla}_k) = W_k + 2\mu\epsilon_k X_k$
- μ controls stability and convergence speed. Training data should be in random order.

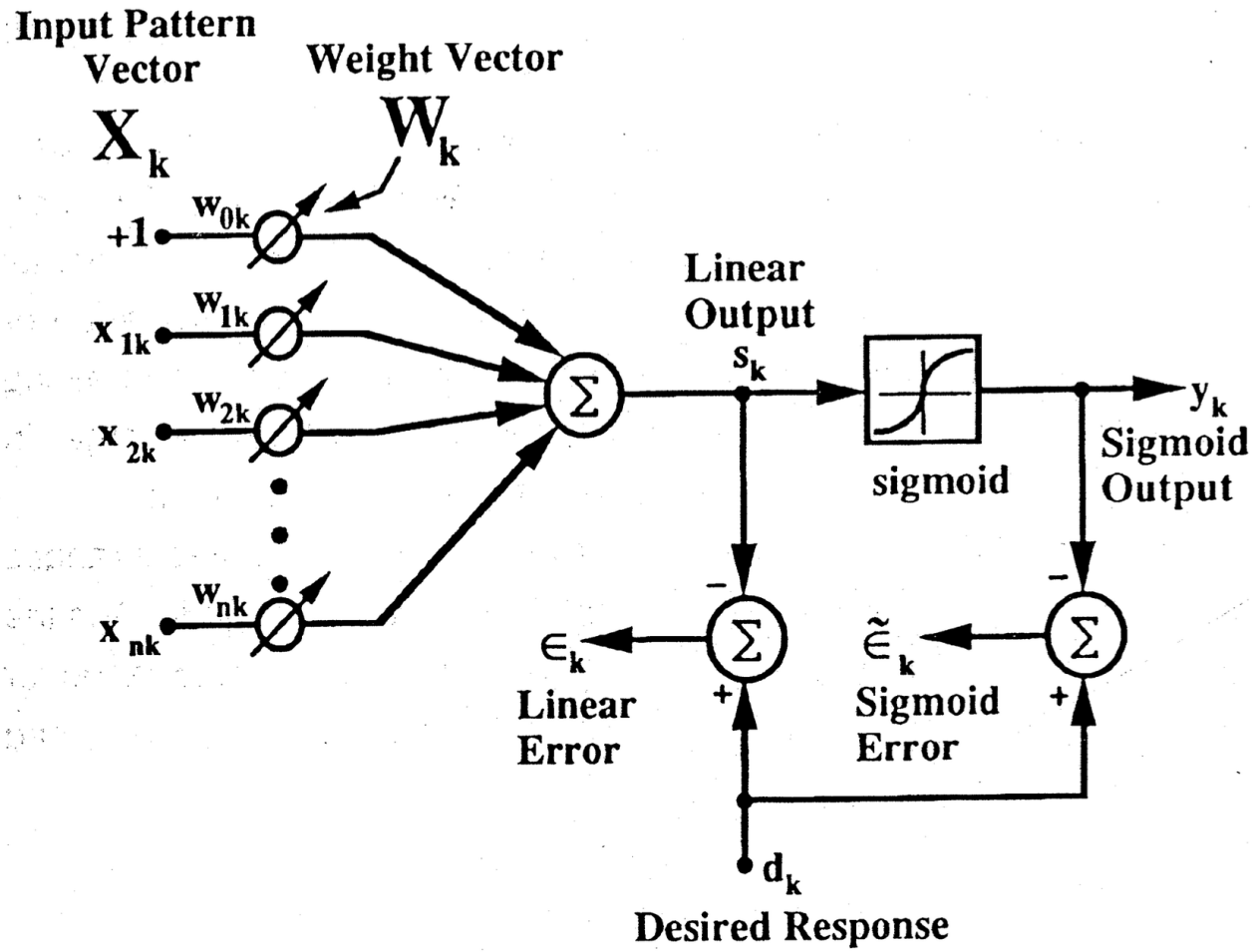
μ-LMS V.S. α-LMS

- α-LMS: $W_{k+1} = W_k + \alpha \frac{\epsilon_k X_k}{|X_k|^2}$, μ-LMS: $W_{k+1} = W_k + 2\mu\epsilon_k X_k$
- α-LMS is self-normalization, with the parameter α determining the fraction of the instantaneous error to be corrected with each adaption.
- μ-LMS is constant-coefficient linear algorithm.
- In practice, α-LMS usually converges faster than μ-LMS.
- μ-LMS converges to minimum MSE solution (Wiener solution), while α-LMS converges to a biased solution.
- Normalized training set: $\widetilde{X}_k \triangleq \frac{X_k}{|X_k|}, \widetilde{d}_k \triangleq \frac{d_k}{|X_k|}$.
- α-LMS achieves minimum MSE solution for normalized training set.

Steepest Descent Rules

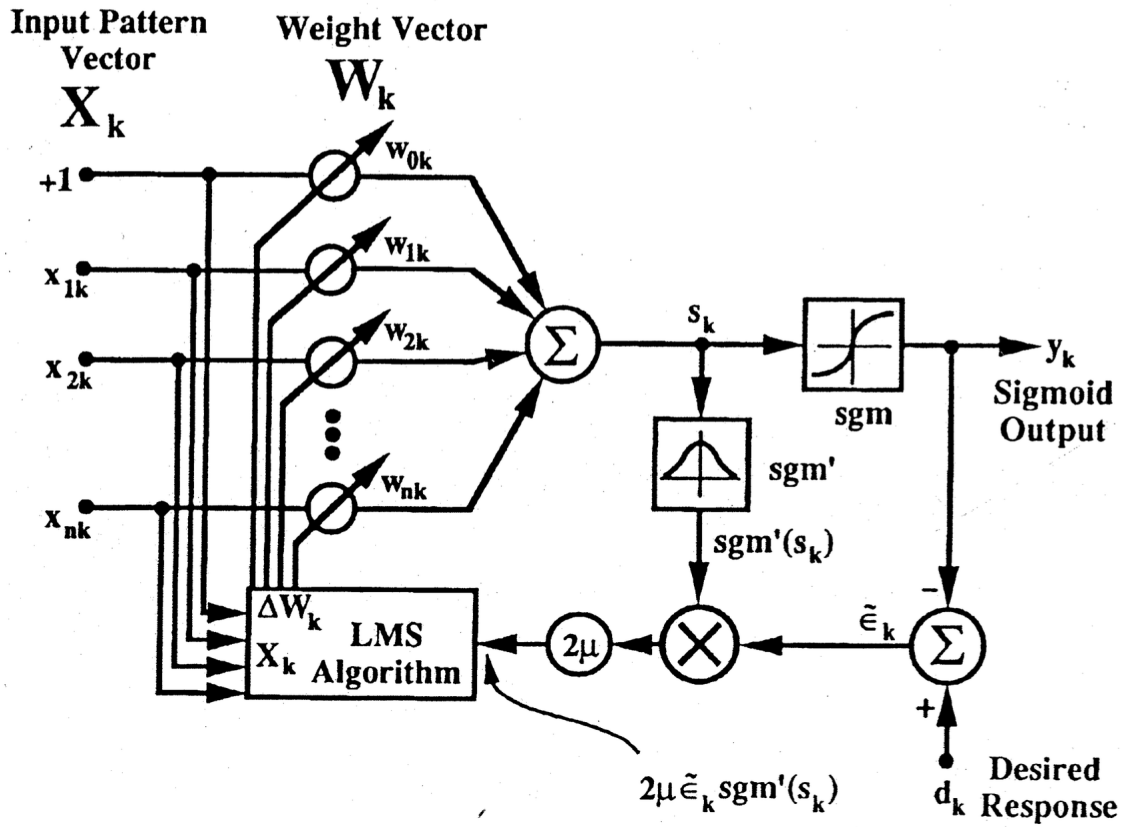
Sigmoid Adaline

- Sigmoid Adaline: extend the Adaline to include the use of a sigmoid in place of the signum.



Backpropagation for Sigmoid Adaline

- $y_k = sgm(s_k)$
- Use instantaneous gradient: $\hat{\nabla}_k = -2\tilde{\epsilon}_k sgm'(s_k)X_k$
- Update weights: $W_{k+1} = W_k + \mu(-\hat{\nabla}_k) = W_k + 2\tilde{\epsilon}_k sgm'(s_k)X_k$



Madaline Rule III (MRIII) for Sigmoid Adaline

- Motivation: the backpropagation algorithm requires accurate implementation of sigmoid function hardware. Need another way to compute the gradient, which does not rely on the accurate function hardware.
- Adding a small perturbation signal Δs to s_k, record the effect on y_k and ϵ_k.
- $\hat{\nabla}_k = \frac{\partial \widetilde{\epsilon}_k^2}{\partial W_k} = \frac{\partial \widetilde{\epsilon}_k^2}{\partial s_k}\frac{\partial s_k}{\partial W_k} = \frac{\partial \widetilde{\epsilon}_k^2}{\partial s_k} X_k \approx \left(\frac{\Delta \widetilde{\epsilon}_k^2}{\Delta s}\right) X_k \approx 2\widetilde{\epsilon}_k \left(\frac{\Delta \widetilde{\epsilon}_k}{\Delta s}\right) X_k$
- $W_{k+1} = W_k - \mu\left(\frac{\Delta \widetilde{\epsilon}_k^2}{\Delta s}\right) X_k$ or

  $Weightsupdate : W_{k+1} = W_k - 2\mu\widetilde{\epsilon}_k \left(\frac{\Delta \widetilde{\epsilon}_k}{\Delta s}\right) X_k$
- Backpropagation and MRIII are mathmatically equivalent if the perturbation Δs is small. MRIII is robust even with the analog implementation.

Backpropagation for Networks

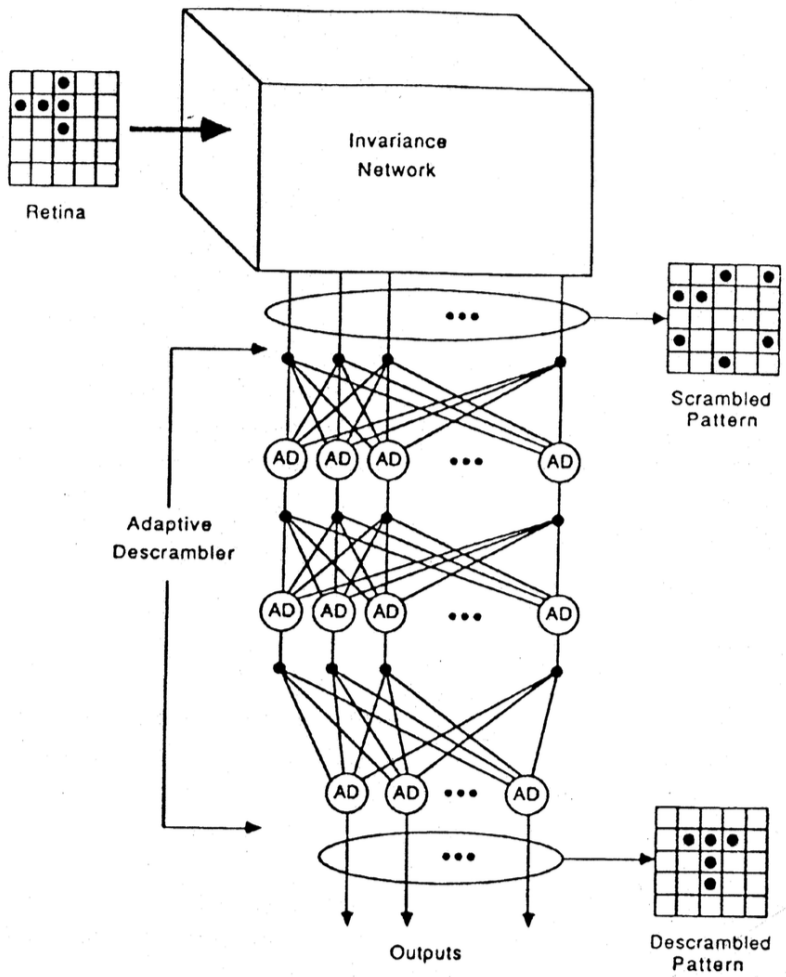
- Intuition: propagate the error backward from the output layer to the first layer
- For an input pattern vector X:
  - Sweep forward through the system to get an output respond vector Y
  - Compute the errors in each output
  - Sweep the effects of the errors backward through the network to associate a "square error derivative" δ with each Adaline
  - Compute a gradient from each δ
  - Update the weights of each Adaline based upon the corresponding gradient
- More details are in the next talk.

MRIII for Networks

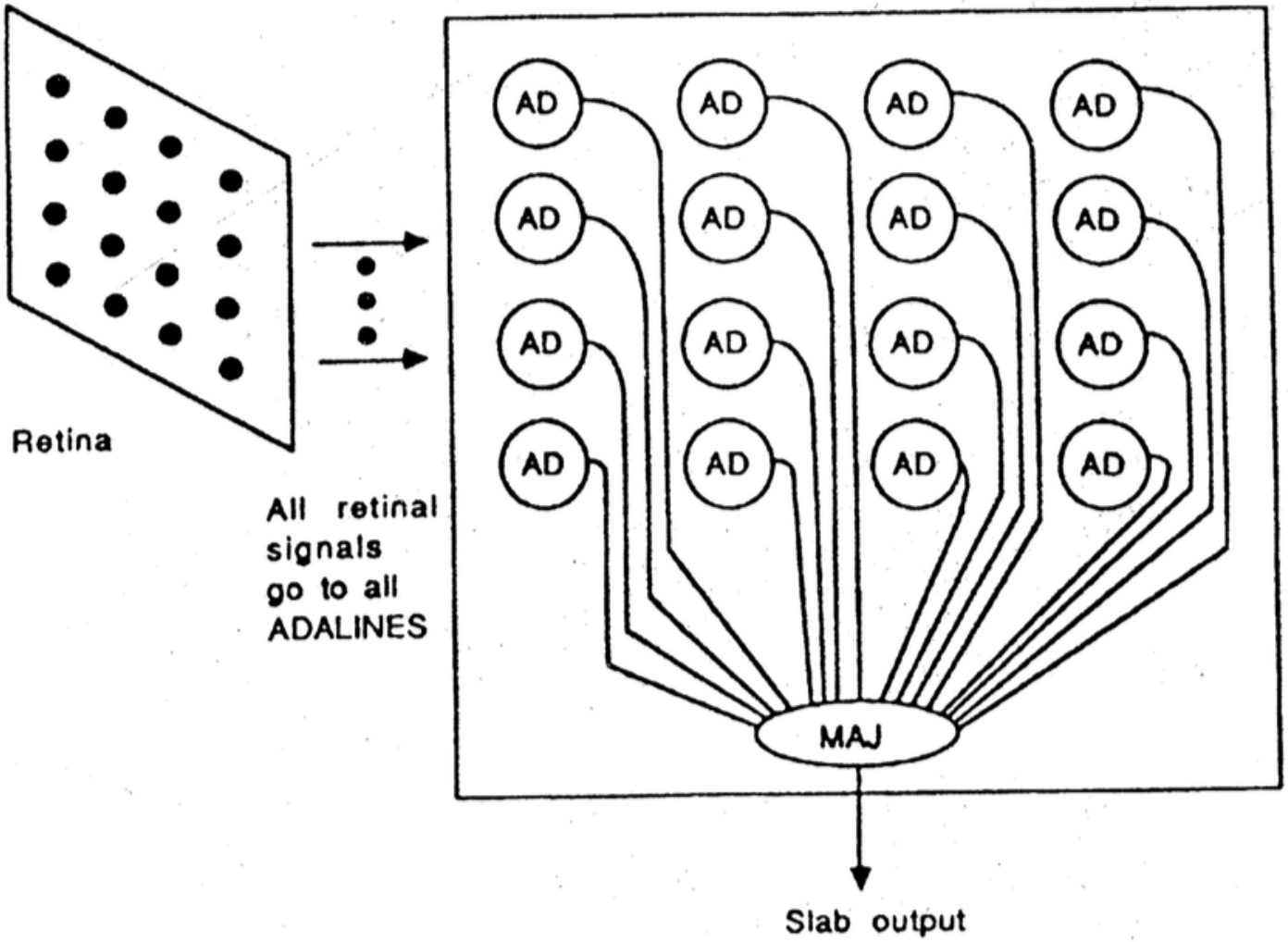
- Same idea as MRIII for sigmoid Madaline
- Measure the sum square output response error
  $\epsilon^2 = (d_1 - y_1)^2 + (d_2 - y_2)^2 = \epsilon_1^2 + \epsilon_2^2$
- $\frac{\Delta(\epsilon^2)}{\Delta s} = \frac{\Delta(\epsilon_1^2+\epsilon_2^2)}{\Delta s} \approx \frac{\partial \epsilon^2}{\partial s}$
- $\hat{\nabla}_k = \frac{\partial \epsilon_k^2}{\partial W_k} = \frac{\partial \epsilon_k^2}{\partial s_k}\frac{\partial s_k}{\partial W_k} = \frac{\partial \epsilon_k^2}{\partial s_k} X_k \approx \frac{\Delta \epsilon_k^2}{\Delta s} X_k$
- $W_{k+1} = W_k - \mu \frac{\Delta \epsilon_k^2}{\Delta s} X_k$

Invariance of Neural Network

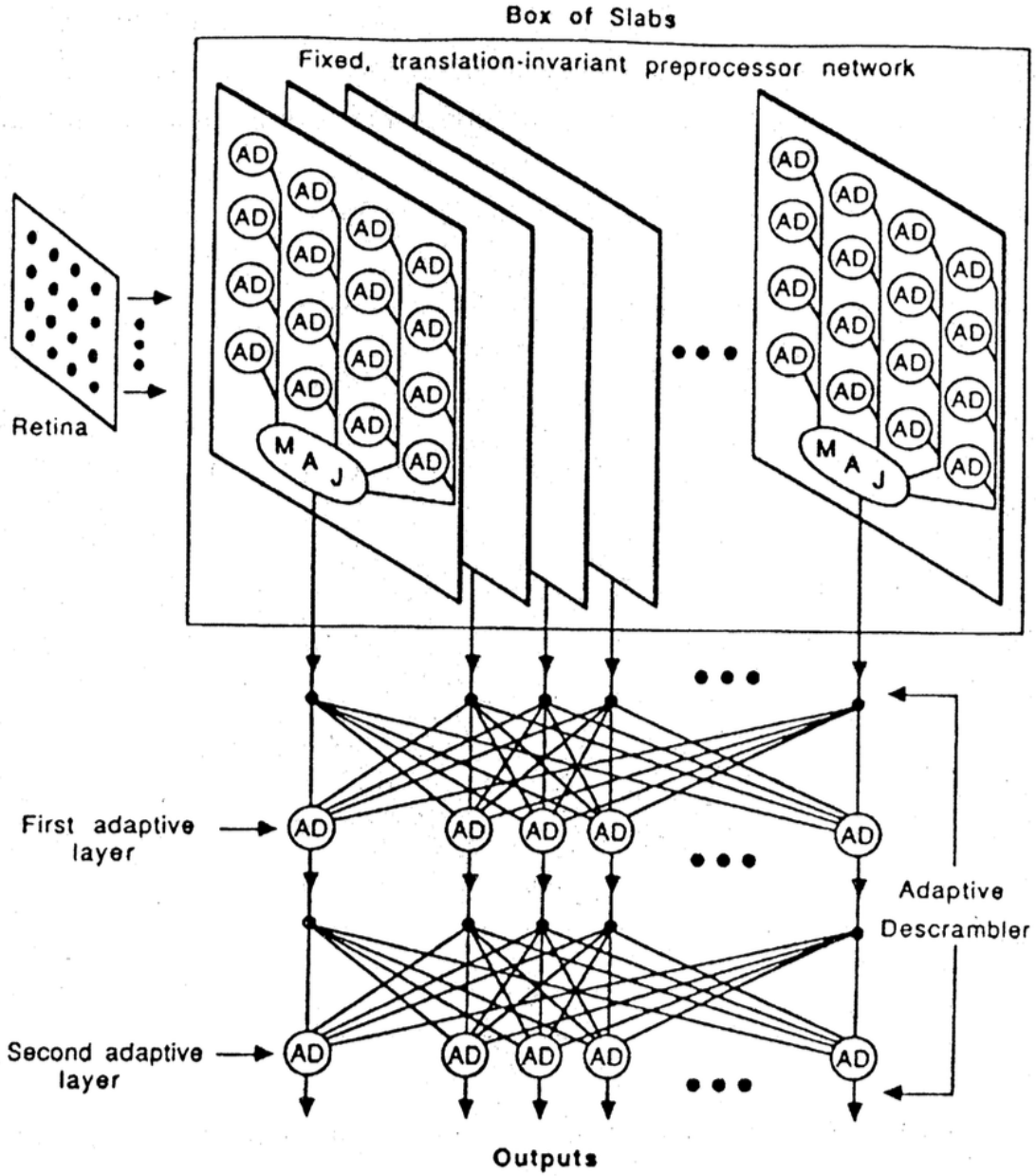
- Neural network should be invariant to translation, rotation and scale change of the input pattern.



Invariance to up-down, left-right translation



Invariance to up-down, left-right translation

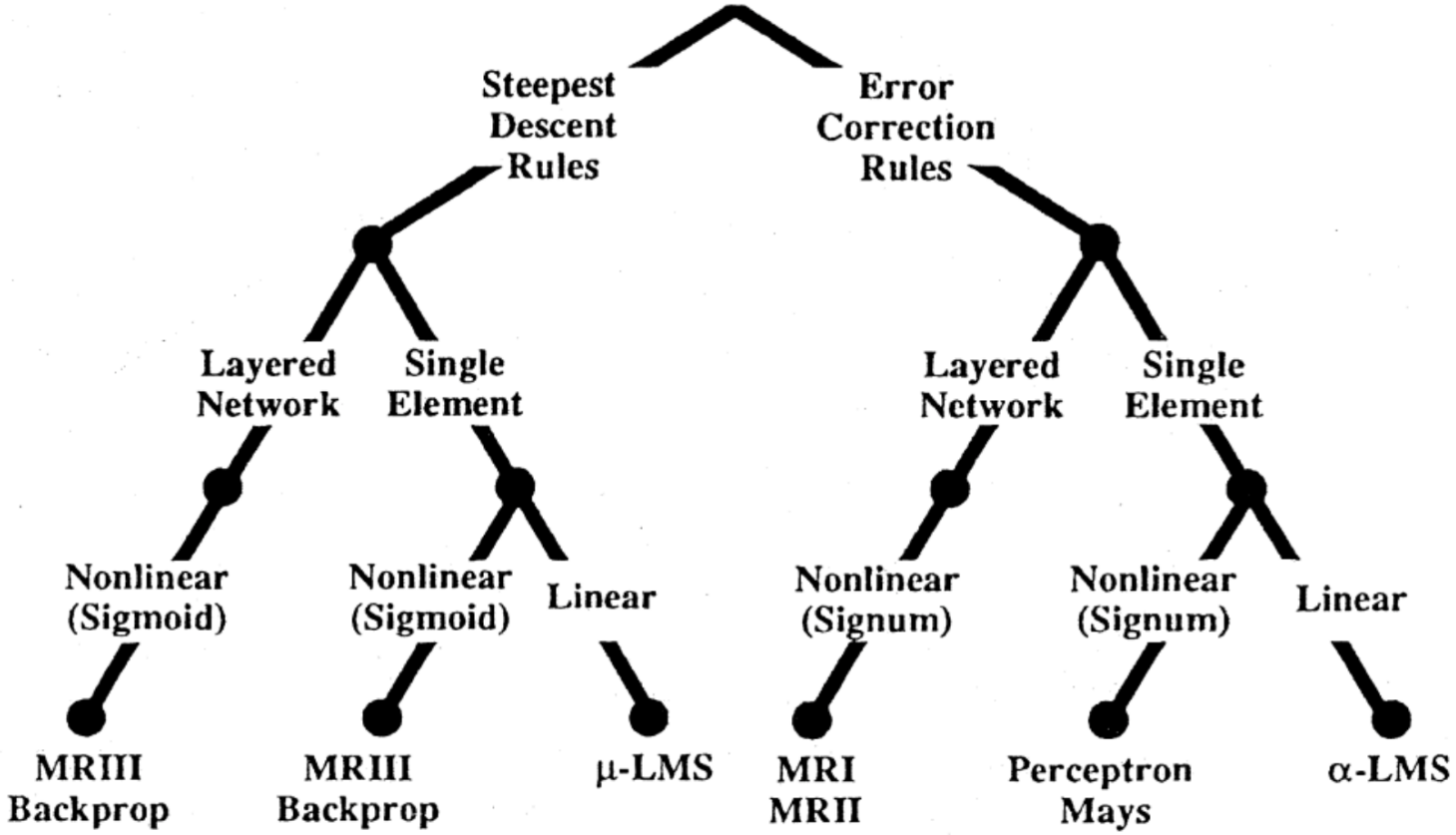


Invariance to up-down, left-right translation

- The roles of the various Adalines interchange.
- The "key" weights (W_1) can be randomly chosen or manufactured.
- the rotation and scale invariance can be obtained in the same way.

$$
\begin{array}{cccc}
(W_1) & T_{R1}(W_1) & T_{R2}(W_1) & T_{R3}(W_1) \\
T_{D1}(W_1) & T_{R1}T_{D1}(W_1) & T_{R2}T_{D1}(W_1) & T_{R3}T_{D1}(W_1) \\
T_{D2}(W_1) & T_{R1}T_{D2}(W_1) & T_{R2}T_{D2}(W_1) & T_{R3}T_{D2}(W_1) \\
T_{D3}(W_1) & T_{R1}T_{D3}(W_1) & T_{R2}T_{D3}(W_1) & T_{R3}T_{D3}(W_1)
\end{array}
$$

Summary


Steepest Descent Rules
Error Correction Rules
Layered Network
Single Element
Layered Network
Single Element
Nonlinear (Sigmoid)
Nonlinear (Sigmoid)
Linear
Nonlinear (Signum)
Nonlinear (Signum)
Linear
MRIII Backprop
MRIII Backprop
μ-LMS
MRI MRII
Perceptron Mays
α-LMS